OOP and Representation

Computer Science Mentors 61A

March 9 – March 13, 2026

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

1 OOP

1. What would Python display in the land of Oz?

```
class Elphaba(object):
    magic = 'Defying Gravity'

    def __init__(self, magic):
        self.magic = magic

    def spell(self):
        return type(self).magic + ' and ' + self.magic

class Glinda(Elphaba):
    magic = 'Popular'

    def __init__(self, magic):
        Elphaba.__init__(self, 'good ' + magic)

elphaba = Elphaba('Wicked')
```

(a) `>>> Elphaba.magic`

(b) `>>> elphaba.magic`

(c) `>>> elphaba.spell()`

(d) `>>> Elphaba.spell()`

(e) `>>> Elphaba.spell(elphaba)`

(f)
```
>>> glinda = Glinda('Power')
>>> Glinda.magic
```

(g) `>>> glinda.magic`

(h) `>>> glinda.spell()`

2. The `DLList` class is a spin off of the normal `Link` class we learned in class; each `DLList` link has a `prev` attribute that keeps track of the previous link and a **`next`** attribute that keeps track of the next link. Fill in the following methods for the `DLList` class.

(a)
```
class DLList:
    """
    >>> lst = DLList(6, DLList(1))
    >>> lst.value
    6
    >>> lst.next.value
    1
    >>> lst.prev.value
    AttributeError: 'NoneType' object has no attribute 'value'
    """
    empty = None
    def __init__(self, value, next=empty, prev=empty):

      ____________________________________

      ____________________________________

      ____________________________________
```

(b)
```
    def add_last(self, value):
      """
      >>> lst = DLList(6)
      >>> lst.add_last(1)
      >>> lst.value
      6
      >>> lst.next.value
      1
      >>> lst.next.prev.value
      6
      """
      pointer = self
      while ____________________________________:

        ________________________________________

      ________________ = DLList(_________________________________)
```

(c)
```
    def add_first(self, value):
      """
      >>> lst = DLList('A')
      >>> lst.add_first(1)
      >>> lst.value
      1
      >>> lst.next.value
      'A'
      >>> lst.next.prev.value
      1
      >>> lst.add_first(6)
      >>> lst.value
      6
```

```
>>> lst.next.next.prev.value
1
"""
old_first = DLList(_____________________________)

_______________ = _____________________________

_______________ = _____________________________

if _____________________________:

  ___________________________________________________
```

3. Let's use OOP to help us implement our good friend, the ping-pong sequence!

   As a reminder, the ping-pong sequence counts up starting from 1 and is always either counting up or counting down.

   At element `k`, the direction switches if `k` is a multiple of 7 or contains the digit 7.

   The first 30 elements of the ping-pong sequence are listed below, with direction swaps marked using brackets at the 7th, 14th, 17th, 21st, 27th, and 28th elements:

   ```
   1 2 3 4 5 6 [7] 6 5 4 3 2 1 [0] 1 2 [3] 2 1 0 [-1] 0 1 2 3 4
   [5] [4] 5 6
   ```

   Assume you have a function `has_seven(k)` that returns True if k contains the digit 7.

   ```
   >>> tracker1 = PingPongTracker()
   >>> tracker2 = PingPongTracker()
   >>> tracker1.next()
   1
   >>> tracker1.next()
   2
   >>> tracker2.next()
   1
   ```

   ```
   class PingPongTracker:
       def __init__(self):




       def next(self):
   ```

2 Representation

1. The Reprtilia class stores the common and scientific names of various reptiles.

   Brief introduction to `__repr__`
   The `__repr__` magic method of objects returns the "official" string representation of an object. You can invoke it directly by calling **`repr`**`(<some` **`object`**`>)`. However, `__repr__` doesn't always return something that is easily readable, that is what `__str__` is for. Rather, `__repr__` ensures that all information about the object is present in the representation. When you ask Python to represent an object in the Python interpreter, it will automatically call **`repr`** on that object and then print out the string that **`repr`** returns.

```
"""
>>> python = Reprtilia("python", "pythonidae")
>>> iguana = Reprtilia("iguana", "iguana")
>>> python
Reptilia('python', 'pythonidae')
>>> f'Did you know that {python}?"
'Did you know that a python is a "pythonidae"?'
"""

class Reprtilia:
    def __init__(self, name, scientific):
        self.name = name
        self.scientific = scientific
```

2. How do we define the `__repr__` method?

```
    def __repr__(self):
```

3. How do we define the `__str__` method?

```
    def __str__(self):
```

4. Your friend, Julian, decides to create a new object class based off of Reprtilia called ReprtiliaNoises, and as such, decides to overhaul your `__repr__` and `__str__`! How would the following be implemented following the doctests?

```
"""
>>> python = ReprtiliaNoises("python", "pythonidae", "hiss")
>>> python
ReprtiliaNoises('python', 'pythonidae', 'hiss')
>>> f"Today's reptile of the day is a {python}!"
"Today's reptile of the day is a python (species: 'pythonidae',
   noise: 'hiss')!"
"""
```

5. How does Julian redefine the `__init__` method?

```
class ReprtiliaNoises(Reprtilia):
    def __init__(self, name, scientific, noise):
```

6. How does Julian redefine the `__repr__` and `__str__` method?

```
    def __repr__(self):
```

7. How does Julian redefine the `__str__` method?

```
    def __str__(self):
```

8. Implement the classes so the following code runs.

```
'''
>>> p = Plant()
>>> p.height
1
>>> p.materials
[]
>>> p.absorb()
>>> p.materials
[|Sugar|]
>>> Sugar.sugars_created
1
>>> p.leaf.sugars_used
0
>>> p.grow()
>>> p.materials
[]
>>> p.height
2
>>> p.leaf.sugars_used
1
'''

class Plant:
    def __init__(self):
        '''A Plant has a Leaf, a list of sugars created so far,
        and an initial height of 1.
        '''
        self.leaf = Leaf(self)
        self.materials = ___________
        self.height = ___________

    def absorb(self):
        '''Calls the Leaf to create sugar.'''









    def grow(self):
        '''A Plant consumes all of its sugars to grow, each of which
        increases its height by 1.
        '''









class Leaf:
```

```
    def __init__(self, plant): # plant is a Plant instance
        '''A Leaf is initially alive, and keeps track of how many
        sugars it has created.
        '''



    def absorb(self):
        '''If this Leaf is alive, a Sugar is added to the plant's
        list of sugars.
        '''
        if self.alive:


    def __repr__(self):
        return '|Leaf|'

class Sugar:
    sugars_created = 0

    def __init__(self, leaf, plant):



    def activate(self):
        '''A sugar is used.'''




    def __repr__(self):
        return '|Sugar|'
```